APOSTILA DO ALUNO — AULA 9

Revisão Integradora UML — os 4 diagramas juntos (TechBurger)

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 15/10/2026 (quinta-feira) | Horário: 07:30 às 10:55

Objetivos desta aula

- Seguir o TechBurger de ponta a ponta: o mesmo sistema nos 4 diagramas, com uma regra de negócio atravessando todos.
- Entender os 4 diagramas UML do curso — casos de uso, classes, atividades e sequência — como vistas do MESMO sistema, cada uma respondendo a uma pergunta.
- Usar a técnica dos grifos para sair do texto do cliente e chegar aos elementos de cada diagrama.
- Aplicar as regras de coerência entre os diagramas (nomes, métodos, cardinalidades, guardas e ordem) e reconhecer as diferenças que NÃO são incoerência.
- Montar uma matriz de rastreabilidade, seguindo cada regra de negócio por todos os diagramas.
- Revisar os conceitos principais de cada diagrama a partir do resultado do exemplo guiado.
- Modelar, em grupo e contra o relógio, a EcoBike nos 4 diagramas (Atividade 1).
- Encontrar incoerências entre um diagrama de classes e um de sequência.

Hoje não vamos aprender um diagrama novo. Vamos juntar os quatro que já sabemos e aprender a fazer com que eles contem a MESMA história.

1. Os 4 diagramas trabalhando juntos

1.1 Um sistema, várias vistas

Uma casa não é descrita por um desenho só. Existe a planta baixa (paredes e portas), o projeto elétrico (tomadas e fios) e o hidráulico (canos e registros). São VISTAS diferentes da mesma casa — e todas precisam concordar: se a tomada do projeto elétrico cai no meio de uma porta da planta baixa, alguém errou, e o erro vai aparecer na obra.

Com a UML acontece a mesma coisa. Cada diagrama mostra um lado do sistema, e nenhum deles, sozinho, conta a história inteira. O diagrama de classes não diz em que ordem as coisas acontecem; o de atividades não diz quais dados existem; o de casos de uso não mostra nenhuma regra por dentro. Juntos — e coerentes entre si — eles viram a especificação que a equipe usa para programar e testar. Incoerentes, eles viram uma fonte de discussões e de bugs: quem programa segue um diagrama, quem testa segue outro, e o sistema não fecha.

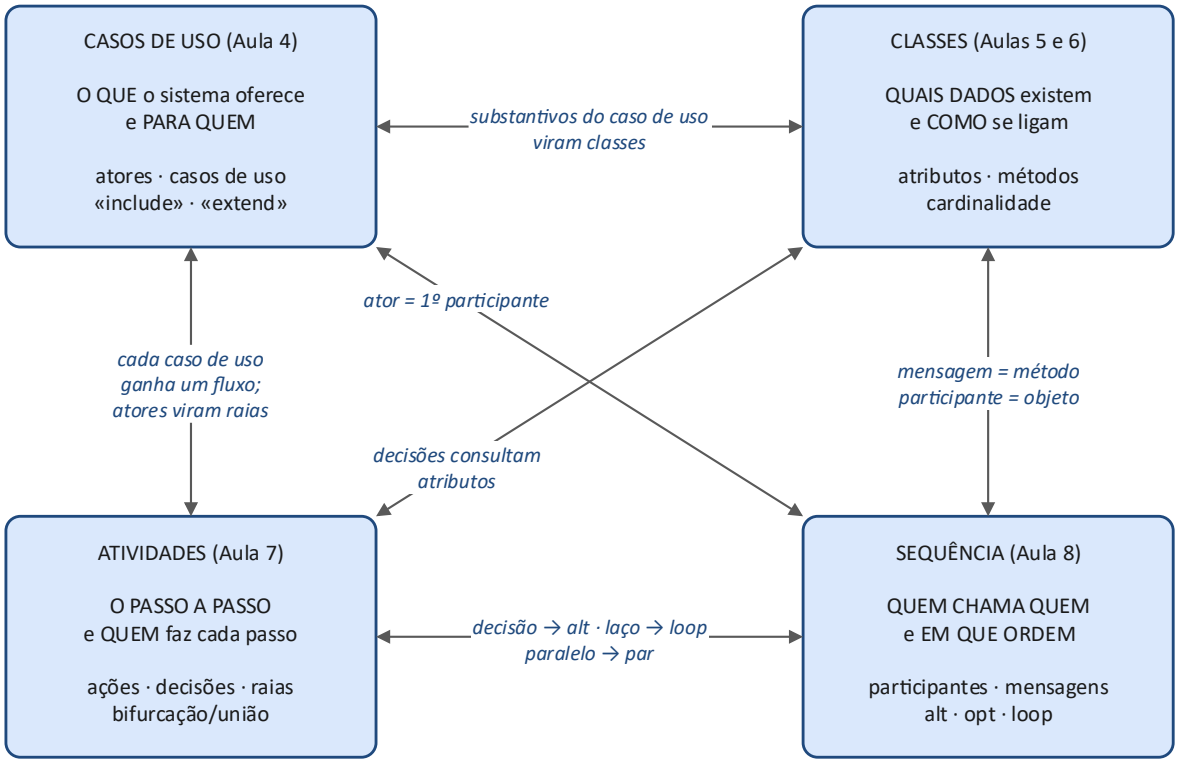


Figura 1 — Os 4 diagramas do curso e como eles se ligam.

1.2 A pergunta de cada diagrama

Diagrama	Pergunta que responde	Elementos principais
Casos de uso (Aula 4)	O QUE o sistema oferece, e PARA QUEM?	Atores, casos de uso, fronteira, «include», «extend», generalização
Classes (Aulas 5 e 6)	QUAIS DADOS existem, e COMO se ligam?	Classes, atributos, métodos, associação, agregação, composição, herança, cardinalidade
Atividades (Aula 7)	Qual o PASSO A PASSO, e QUEM faz cada passo?	Nós inicial e final, ações, decisões com guardas, junção, bifurcação/união, raias
Sequência (Aula 8)	QUEM CHAMA QUEM, e em que ORDEM?	Participantes, linhas de vida, ativações, mensagens, retornos, alt/opt/loop

Por trás dos quatro estão as regras de negócio (Aula 1) e os requisitos e critérios de aceite (Aulas 2 e 3). Os diagramas são uma forma de DESENHAR essas regras — por isso uma regra que não aparece em nenhum diagrama é um sinal de alerta.

1.3 Estrutural x comportamental

O diagrama de classes é ESTRUTURAL: mostra o que o sistema É e o que ele GUARDA — uma fotografia do sistema parado. Costuma existir um só para o sistema todo, e ele muda pouco com o tempo. Os outros três são COMPORTAMENTAIS: mostram o que o sistema FAZ — o filme do sistema em ação. Deles existem vários: um diagrama de atividades por processo importante, um ou mais diagramas de sequência por caso de uso (um por cenário).

1.4 Em que ordem trabalhar

1. Ler o texto do cliente e listar as regras de negócio (Aula 1).
2. Escrever os requisitos funcionais e não funcionais e os critérios de aceite (Aulas 2 e 3).
3. Desenhar os casos de uso: quem usa o sistema, e para quê (Aula 4).
4. Desenhar as classes, com atributos, métodos e cardinalidade (Aulas 5 e 6).
5. Para os casos de uso principais, desenhar as atividades (Aula 7) e a sequência (Aula 8).
6. Conferir a coerência entre todos — e voltar para corrigir o que for preciso.

Essa ordem é um ponto de partida, não uma lei. Na prática a modelagem é ITERATIVA, um vai e volta: ninguém acerta o diagrama de classes de primeira — quase sempre é a sequência que mostra o método que faltou. A atividade revela uma decisão que não estava nas regras, e aí é preciso voltar e perguntar ao cliente. Cada diagrama novo é também uma revisão dos anteriores.

2. Do texto ao modelo: a técnica dos grifos

O ponto de partida de qualquer modelagem é o texto do cliente — a entrevista, o e-mail, o documento de requisitos. Uma técnica simples, usada desde a Aula 1, ajuda a não esquecer nada: ler o texto com canetas de cores diferentes (ou, no computador, com negrito e sublinhado) e marcar cada tipo de palavra.

2.1 O que grifar e no que cada marca vira

O que marcar no texto	Exemplo	Vira...
Substantivos	cliente, pedido, pagamento, entrega	Classes, atributos e atores
Verbos	fazer pedido, confirmar, cancelar	Casos de uso, ações e métodos
SE / SENÃO / QUANDO / EXCETO	se o pagamento for recusado...	Decisões (atividades); alt ou opt (sequência); exceção da regra
PARA CADA / ATÉ / REPETE	para cada item...	Laço (atividades); loop (sequência)
AO MESMO TEMPO / ENQUANTO	enquanto a cozinha prepara...	Bifurcação e união; par ou mensagem assíncrona
Números	R$ 15,00, 5 km, 10 itens	Cardinalidades, guardas e valores das regras

2.2 Exemplo: os grifos no texto do TechBurger

Relembrem o texto da gerente Fernanda (Aula 1): “Pedidos feitos pelo aplicativo só podem ser confirmados quando o valor total for igual ou superior a R$ 15,00. Assim que o pagamento de um pedido é confirmado, o pedido é enviado para a tela da cozinha. Pagamentos via PIX feitos pelo aplicativo recebem 5% de desconto. A loja só realiza entrega em endereços a até 5 km. Pedidos com mais de 10 itens exigem confirmação manual de um gerente.”

Trecho grifado	Onde aparece nos diagramas
“pedidos do APLICATIVO só confirmam com VALOR TOTAL de R$ 15,00 ou mais”	Atributos canal e /valorTotal (Pedido); restrição { canal = app implica valorTotal >= 15,00 }; decisão [valor total ≥ R$ 15,00?]
“ASSIM QUE o PAGAMENTO é CONFIRMADO, o pedido vai	Fazer Pedido pelo App «include» Confirmar Pagamento;

para a COZINHA”	decisão [pagamento confirmado?] (atividades) e alt [resultado = aprovado] (sequência)
“PIX pelo app tem 5% de DESCONTO”	Aplicar Desconto PIX «extend» Confirmar Pagamento; método aplicarDesconto() de Pagamento
“ENTREGA só ATÉ 5 KM”	Pedido 1 —— 0..1 Entrega; método validarRaio(); decisão [endereço a até 5 km?]
“MAIS DE 10 ITENS exigem confirmação do GERENTE”	Ator Gerente e caso de uso Autorizar Pedido Grande; decisão [mais de 10 itens?] e opt [mais de 10 itens]

Quiz rápido: para onde vai?

Para cada trecho, digam em que elemento de quais diagramas ele se transforma. Respostas no Gabarito.

1. “Cliente” (a pessoa que usa o app).
2. “cada pedido pode ter uma entrega”.
3. “se o pagamento for recusado, cancela o pedido”.
4. “para cada item do carrinho, confere o estoque”.
5. “o sistema calcula o valor total do pedido”.
6. “a cozinha prepara enquanto o sistema credita os pontos”.

3. Exemplo resolvido: TechBurger de ponta a ponta

O TechBurger apareceu em todas as aulas, sempre em um diagrama de cada vez: os casos de uso na Aula 4, as classes nas Aulas 5 e 6, as atividades do pedido pelo app na Aula 7 e a sequência do mesmo pedido na Aula 8. Agora vamos olhá-lo inteiro, com os quatro diagramas lado a lado — e seguir uma regra atravessando todos eles.

3.1 O fio condutor: só vai para a cozinha com pagamento confirmado

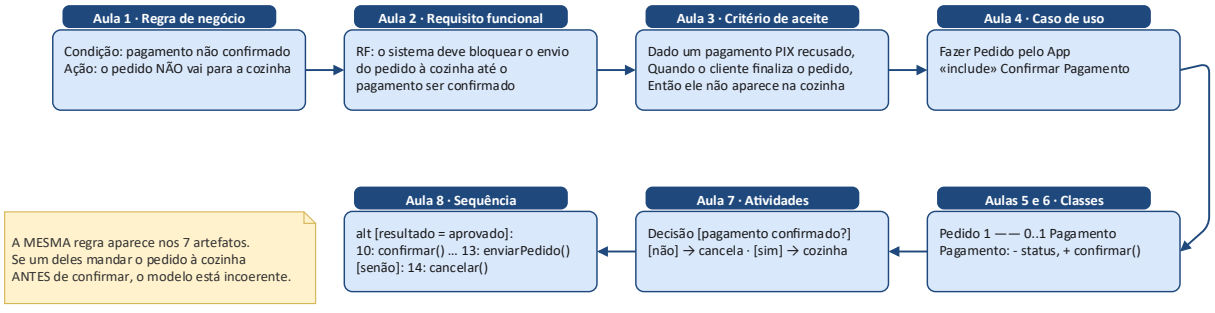


Figura 2 — A regra do pagamento atravessando os 7 artefatos das Aulas 1 a 8.

A regra nasce no texto do cliente (Aula 1): pedido sem pagamento confirmado nunca aparece na cozinha. Ela vira um requisito funcional (“o sistema deve bloquear o envio do pedido à cozinha até que o pagamento seja confirmado”) e um critério de aceite (Aulas 2 e 3). No diagrama de casos de uso ela aparece como «include»: todo Fazer Pedido pelo App passa, obrigatoriamente, por Confirmar Pagamento. No diagrama de classes vira a associação Pedido 1 —— 0..1 Pagamento (o pedido nasce antes do pagamento), o atributo status e o método

confirmar() de Pagamento. Na atividade vira a decisão [pagamento confirmado?], desenhada ANTES de “Envia o pedido para a Cozinha”; na sequência, o alt [resultado = aprovado], que envolve 10: confirmar() … 13: enviarPedido(pedido), com 14: cancelar() no [senão]. Se um desses artefatos mandar o pedido à cozinha antes de confirmar o pagamento, o modelo está INCOERENTE.

3.2 Casos de uso

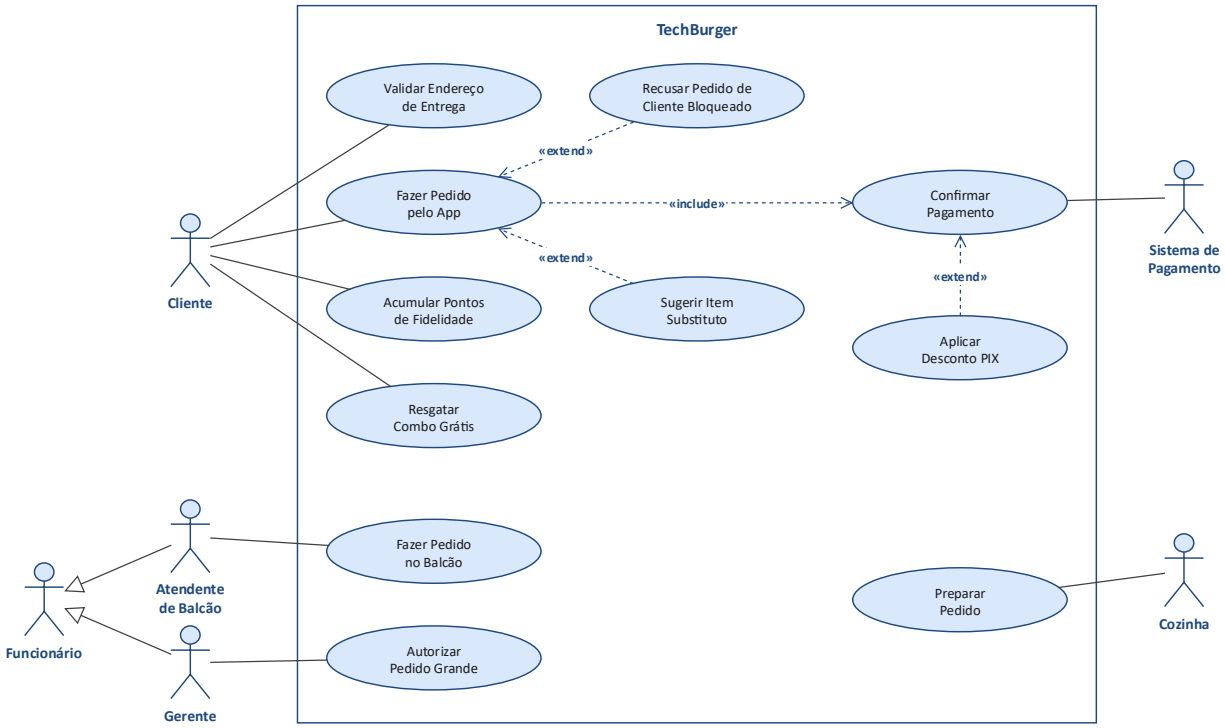


Figura 3 — TechBurger: diagrama de casos de uso (Aula 4).

Atores: Cliente, Atendente de Balcão e Gerente — os dois últimos são tipos de Funcionário (generalização) —, Cozinha e Sistema de Pagamento (ator externo). Fazer Pedido pelo App «include» Confirmar Pagamento, porque todo pedido pelo app passa pela confirmação. Aplicar Desconto PIX «extend» Confirmar Pagamento, porque só acontece quando o cliente escolhe PIX; Sugerir Item Substituto e Recusar Pedido de Cliente Bloqueado «extend» Fazer Pedido pelo App, porque só acontecem em certas situações. Cada caso de uso importante vai ganhar um fluxo no diagrama de atividades e um ou mais cenários no diagrama de sequência — e os NOMES dos atores precisam se repetir, iguais, nas raias e nos participantes.

3.3 Classes

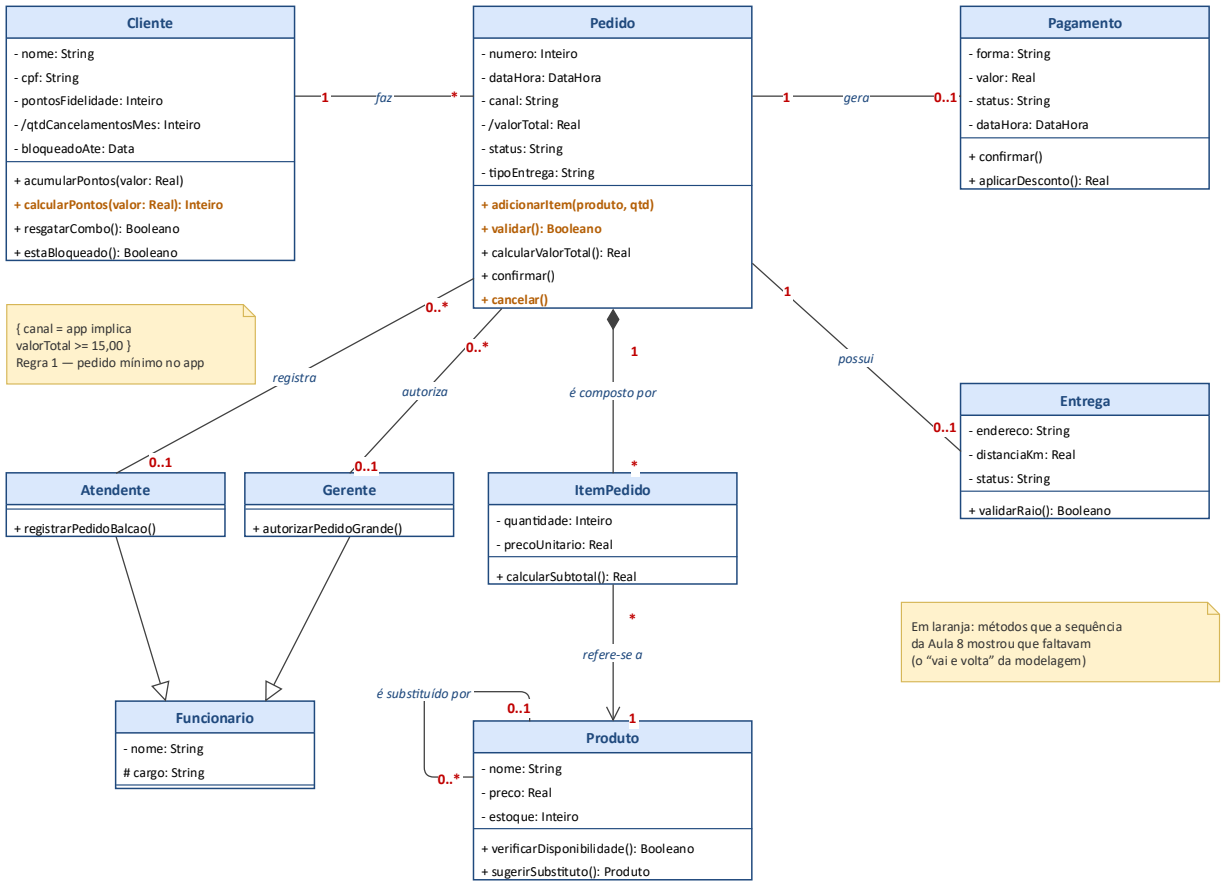


Figura 4 — TechBurger: diagrama de classes com cardinalidade (Aulas 5 e 6); em laranja, os métodos descobertos na Aula 8.

Cliente 1 —— * Pedido e Pedido 1 ◆—— * ItemPedido (composição) são a espinha do modelo. Pedido 1 —— 0..1 Pagamento, porque o pedido é criado antes de o pagamento ser confirmado; Pedido 1 —— 0..1 Entrega, porque pedido de retirada no balcão não tem entrega. O /valorTotal é derivado: é a soma dos ItemPedido, calculada por calcularValorTotal(). Reparem nos métodos em laranja — adicionarItem(), validar() e cancelar() no Pedido, calcularPontos() no Cliente: eles não estavam no diagrama da Aula 5; foi a sequência da Aula 8 que mostrou que faltavam. É o "vai e volta" da seção 1.4 acontecendo de verdade.

3.4 Atividades e sequência: o mesmo pedido pelo app

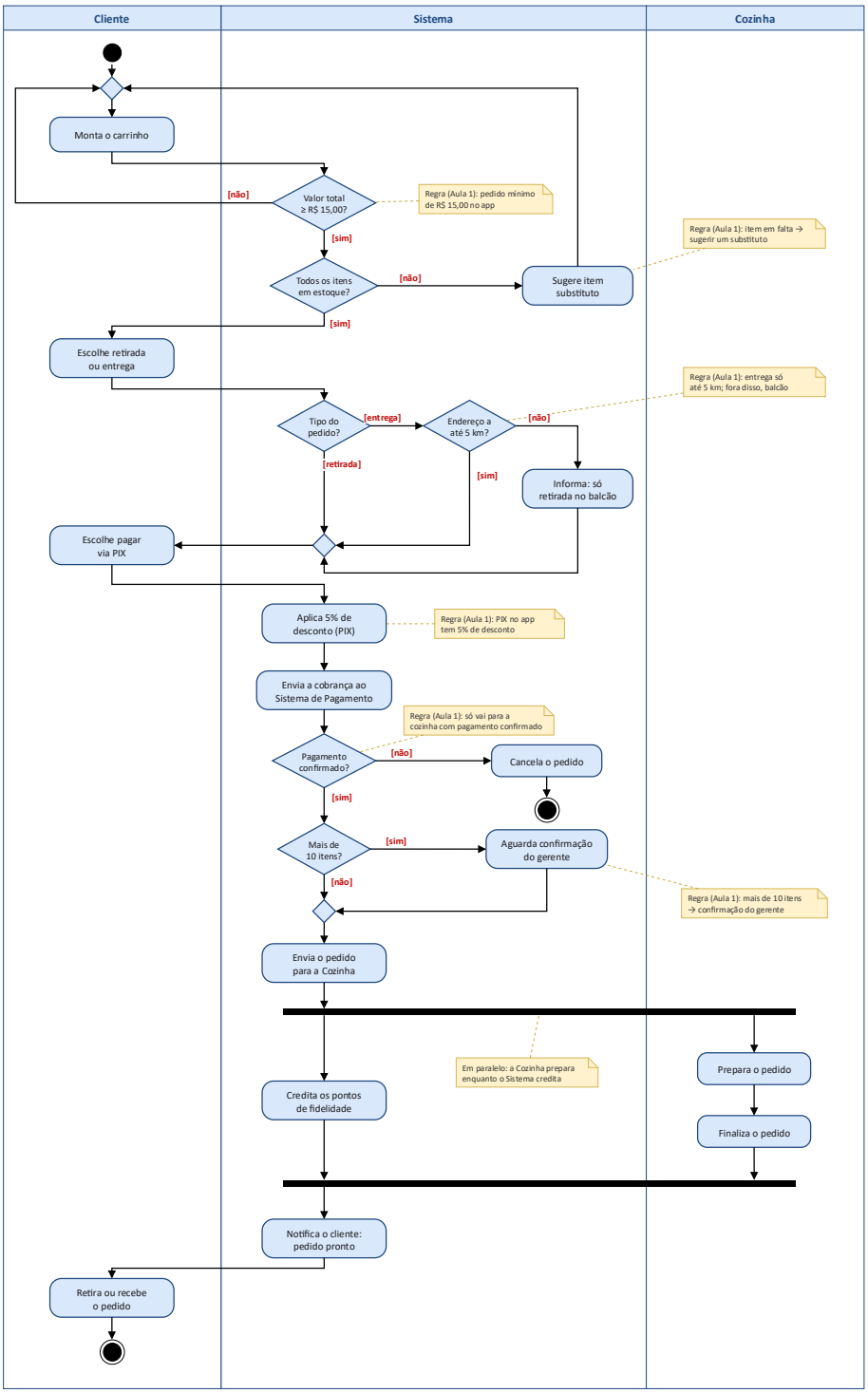


Figura 5 — TechBurger: diagrama de atividades “Fazer pedido pelo app” (Aula 7).

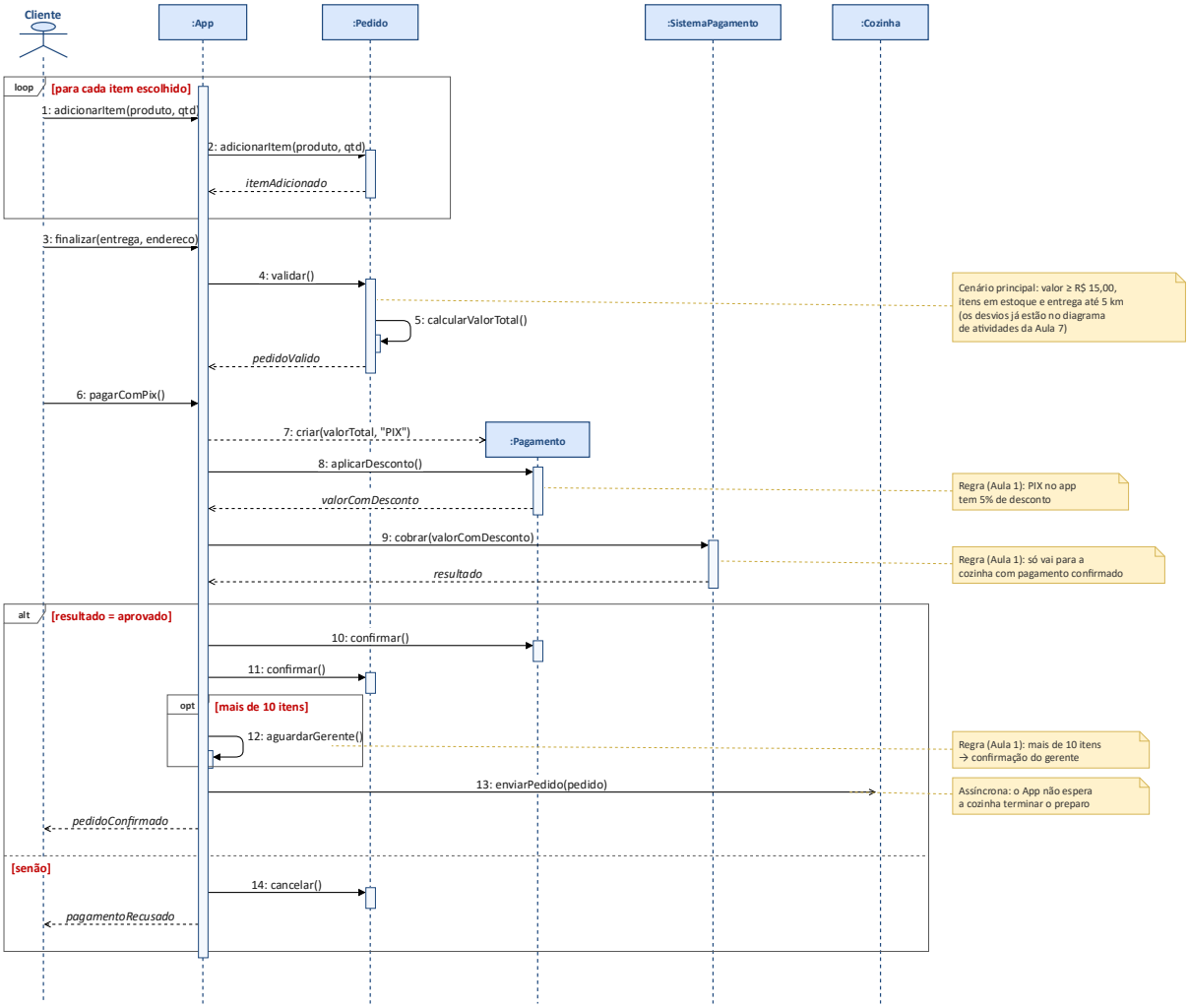


Figura 6 — TechBurger: diagrama de sequência "Pedido pelo app com PIX" (Aula 8).

No diagrama de atividades (Aula 7)	No diagrama de sequência (Aula 8)
Raias: Cliente, Sistema e Cozinha	Ator Cliente e os participantes :App, :Pedido, :Pagamento, :SistemaPagamento e :Cozinha
Decisão [pagamento confirmado?]	9: cobrar(valorComDesconto) e o alt [resultado = aprovado] / [senão]
Decisão [mais de 10 itens?] → Aguarda confirmação do gerente	opt [mais de 10 itens] com 12: aguardarGerente()
Bifurcação: a Cozinha prepara enquanto o Sistema credita os pontos	13: enviarPedido(pedido) ASSÍNCRONA — o :App não espera o preparo

3.5 Nem toda diferença é incoerência

Comparando os dois diagramas acima, aparecem diferenças — e nenhuma delas é erro. A atividade tem seis decisões (valor mínimo, estoque, tipo do pedido, raio de 5 km, pagamento e 10 itens); a sequência passa só por duas, porque mostra UM cenário: pedido para entrega, pago com PIX, acima de R$ 15,00 e com tudo em estoque. A atividade mostra TODOS os caminhos; é esperado que a sequência tenha menos. A atividade tem ações humanas ("Monta o carrinho", "Prepara o pedido"), e a sequência mostra só as chamadas entre objetos do sistema. A bifurcação da atividade (a Cozinha prepara enquanto o Sistema credita os pontos) aparece na

sequência como a mensagem ASSÍNCRONA para a :Cozinha: duas formas de mostrar a mesma ideia. Resumindo: incoerência é quando um diagrama diz uma coisa DIFERENTE do outro. Mostrar MENOS não é incoerência; mostrar DIFERENTE é.

3.6 Mini-exercício: sugerir um item substituto nos 4 diagramas

Em duplas (3 minutos): pensem na regra "quando um item do cardápio está em falta no estoque, o sistema sugere um item substituto antes de permitir a finalização do pedido" (Aula 1). Onde ela aparece em cada um dos 4 diagramas do TechBurger? Uma linha por diagrama basta. Resposta no Gabarito.

4. Coerência e rastreabilidade

4.1 Regras de coerência: os NOMES

- O ator do caso de uso é a raia da atividade e o primeiro participante da sequência — com o MESMO nome.
- Todo participante ":X" da sequência é uma classe X do diagrama de classes. A exceção é o :App (ou :Sistema), que faz o papel da tela e costuma não aparecer como classe de dados.
- Toda mensagem recebida por ":X" é um método da classe X — com a mesma grafia.
- Todo atributo usado numa guarda existe na classe: [valor total ≥ R$ 15,00?] precisa de - /valorTotal: Real no Pedido.
- As cardinalidades batem com as regras: "no máximo um pagamento por pedido" = Pedido 1 —— 0..1 Pagamento.

4.2 Regras de coerência: o COMPORTAMENTO

- Cada decisão da atividade vira um alt ou opt na sequência — quando o cenário passa por ela.
- O laço vira loop; a bifurcação e a união viram par ou mensagem assíncrona.
- «include» (sempre acontece) vira passos que SEMPRE aparecem no fluxo; «extend» (só às vezes) vira um trecho opcional — um opt.
- A ORDEM das regras é a mesma nos dois: validar antes de cobrar, cobrar antes de entregar.
- Nenhuma regra de negócio pode sumir no caminho de um diagrama para outro.

4.3 Tabela de equivalências

Num diagrama...	...aparece no outro como
Ator (casos de uso)	Raia (atividades) e primeiro participante (sequência)
Substantivo importante do texto	Classe ou atributo (classes) e participante (sequência)
Caso de uso	Um fluxo (atividades) e um ou mais cenários (sequência)
Regra com número	Cardinalidade (classes) ou guarda (atividades e sequência)
Decisão (atividades)	alt ou opt (sequência)
«extend» (casos de uso)	Caminho opcional (atividades) ou opt (sequência)
Ação do Sistema (atividades)	Mensagem = método de uma classe (sequência e classes)

4.4 Matriz de rastreabilidade

Como ter certeza de que nenhuma regra ficou para trás? Com uma MATRIZ DE RASTREABILIDADE: uma tabela com uma linha por regra e uma coluna por artefato (requisito, caso de uso, classes, atividades, sequência). Em cada célula escrevemos ONDE a regra aparece naquele diagrama, com o nome exato do elemento. Uma célula

vazia é uma LACUNA: ou a regra sumiu naquele diagrama (e é preciso corrigir), ou ela realmente não se aplica ali (e é preciso dizer por quê). É a mesma ferramenta que as empresas usam para provar que todo requisito foi modelado e testado.

Regra	RF	Caso de uso	Classes	Atividades	Sequência
Só vai para a cozinha com pagamento confirmado	O sistema deve bloquear o envio do pedido à cozinha até o pagamento ser confirmado	Confirmar Pagamento («include» de Fazer Pedido pelo App)	Pedido 1 —— 0..1 Pagamento; Pagamento.status e confirmar()	[pagamento confirmado?]	alt [resultado = aprovado] antes de 13: enviarPedido()
Mais de 10 itens exigem o gerente	O sistema deve exigir a confirmação de um gerente para pedidos com mais de 10 itens	Autorizar Pedido Grande (ator Gerente)	Gerente 0..1 —— 0..* Pedido; autorizarPedidoGrande()	[mais de 10 itens?]	opt [mais de 10 itens] com 12: aguardarGerente()

4.5 Erros de coerência mais comuns

- O mesmo papel com nomes diferentes: Atendente de Balcão no caso de uso, Caixa na atividade.
- Mensagem que não é método da classe: validarEndereco() chegando num :Entrega que só tem validarRaio().
- Participante na sequência que não existe no diagrama de classes (um :Cardapio inventado).
- Cardinalidade que contradiz a regra: Pedido 1 —— 1 Entrega, quando pedido de retirada não tem entrega.
- Guarda invertida (mandar à cozinha justamente quando o pagamento é recusado) ou usando um atributo que não existe.
- Ordem diferente: a atividade valida antes de cobrar, a sequência cobra antes de validar.

Quiz rápido: coerente ou incoerente?

Respostas no Gabarito.

1. Caso de uso com o ator Gerente; na atividade, a raia se chama Supervisor.

2. A sequência envia validarRaio() à :Entrega; a classe Entrega tem + validarRaio().

3. A atividade tem 6 decisões; a sequência do cenário “pedido aprovado” tem só 1 alt e 1 opt.

4. Pedido 1 —— 0..1 Pagamento; a sequência cria DOIS :Pagamento para o mesmo pedido.

5. A atividade confere o estoque antes de cobrar; a sequência cobra e depois confere.

4.6 O nível de detalhe e o trabalho em equipe

Os diagramas precisam estar no MESMO nível de detalhe. Se a atividade tem 20 ações e a sequência do mesmo processo tem 3 mensagens, algo ficou de fora (ou sobrou). Nas classes, entram só as que aparecem nos casos de uso e nas regras — sem inventar. Nas atividades e na sequência, entram as regras e as integrações importantes, não cada clique.

Em equipe, a organização faz toda a diferença: um arquivo .drawio só, com uma página por diagrama; os nomes combinados ANTES de desenhar, numa lista na primeira página (atores, classes, métodos); os diagramas divididos entre as pessoas, mas com revisão cruzada — quem fez a sequência revisa as classes, e vice-versa. No fim, a matriz de rastreabilidade serve de checklist. É exatamente assim que vai funcionar no Projeto Integrador (Aulas 16 a 19).

5. Exemplo guiado: resgatar combo grátis

Vamos construir juntos, ao vivo, no draw.io, o que acontece quando o cliente traz uma regra NOVA. Hoje, pela Regra 2 da Aula 1, ao chegar a 100 pontos o cliente ganha o combo grátis AUTOMATICAMENTE na próxima compra, e os pontos são zerados. A gerente Fernanda quer mudar isso:

- O cliente escolhe QUANDO resgatar o combo grátis: no carrinho, ao fazer um pedido pelo app.
- Continua valendo: só resgata quem tem 100 pontos ou mais, e o resgate zera os pontos.
- Regra NOVA: só 1 combo grátis por pedido. O combo entra no pedido com preço R$ 0,00.

A pergunta é: o que muda em cada um dos 4 diagramas? Vamos responder em 8 passos (slides 48 a 55: em cada um, o que entra no diagrama aparece em laranja).

5.1 Passo a passo

Passo 0 — Ponto de partida: abram o “Aula 9 - Exemplo Guiado (ponto de partida).drawio”. A página Atividades já tem as raias Cliente e Sistema; a página Sequência já tem o boneco Cliente, :App, :Pedido e :Cliente.

Passo 1 — Casos de uso e classes: Resgatar Combo Grátis JÁ existe desde a Aula 4, ligado ao ator Cliente — nada muda. Nas classes, a regra nova vira métodos do Pedido: + temComboGratis(): Booleano (verdadeiro se o pedido já tem um combo grátis) e + adicionarComboGratis() (inclui o combo com preço R$ 0,00). A regra dos 100 pontos vira + podeResgatar(): Booleano no Cliente (verdadeiro se pontosFidelidade ≥ 100). resgatarCombo() já existia no Cliente (Aula 5) e é ele que zera os pontos.

Passo 2 — Atividades, o começo: “Start” no topo da raia Cliente e, logo abaixo, a ação “Pede o combo grátis no carrinho”, ligadas por uma seta.

Passo 3 — 1ª decisão, a regra nova: na raia Sistema, um losango “Condition” com “O pedido já tem um combo grátis?”. Saída [sim]: ação “Informa: só 1 combo grátis por pedido” e nó final. Uma nota amarela com a regra nova fica ligada ao losango por uma linha tracejada.

Passo 4 — 2ª decisão, os 100 pontos: a saída [não] da 1ª decisão leva ao losango “Tem 100 pontos ou mais?”. Saída [não]: “Informa quantos pontos faltam” e nó final. Outra nota amarela com a Regra 2 da Aula 1.

Passo 5 — O caminho [sim]: “Zera os pontos do cliente” → “Adiciona o combo (R$ 0,00) ao pedido” → de volta à raia Cliente, “Vê o combo grátis no carrinho” → nó final. O diagrama fica com três caminhos: já tem combo, faltam pontos, resgatou.

Passo 6 — Sequência, primeiro o app PERGUNTA: 1: pedirComboGratis() do Cliente ao :App; 2: temComboGratis() do :App ao :Pedido (retorno temCombo); 3: podeResgatar() do :App ao :Cliente (retorno pode). Cada mensagem é um método da classe que a recebe (Passo 1).

Passo 7 — O alt: moldura alt [temCombo = falso e pode = verdadeiro] com 4: resgatarCombo() no :Cliente (retorno resgatou) e 5: adicionarComboGratis() no :Pedido (retorno comboAdicionado, que o :App devolve ao cliente). Abaixo da linha tracejada, [senão]: o :App devolve motivoRecusa ao cliente. As duas decisões da atividade viraram UM alt.

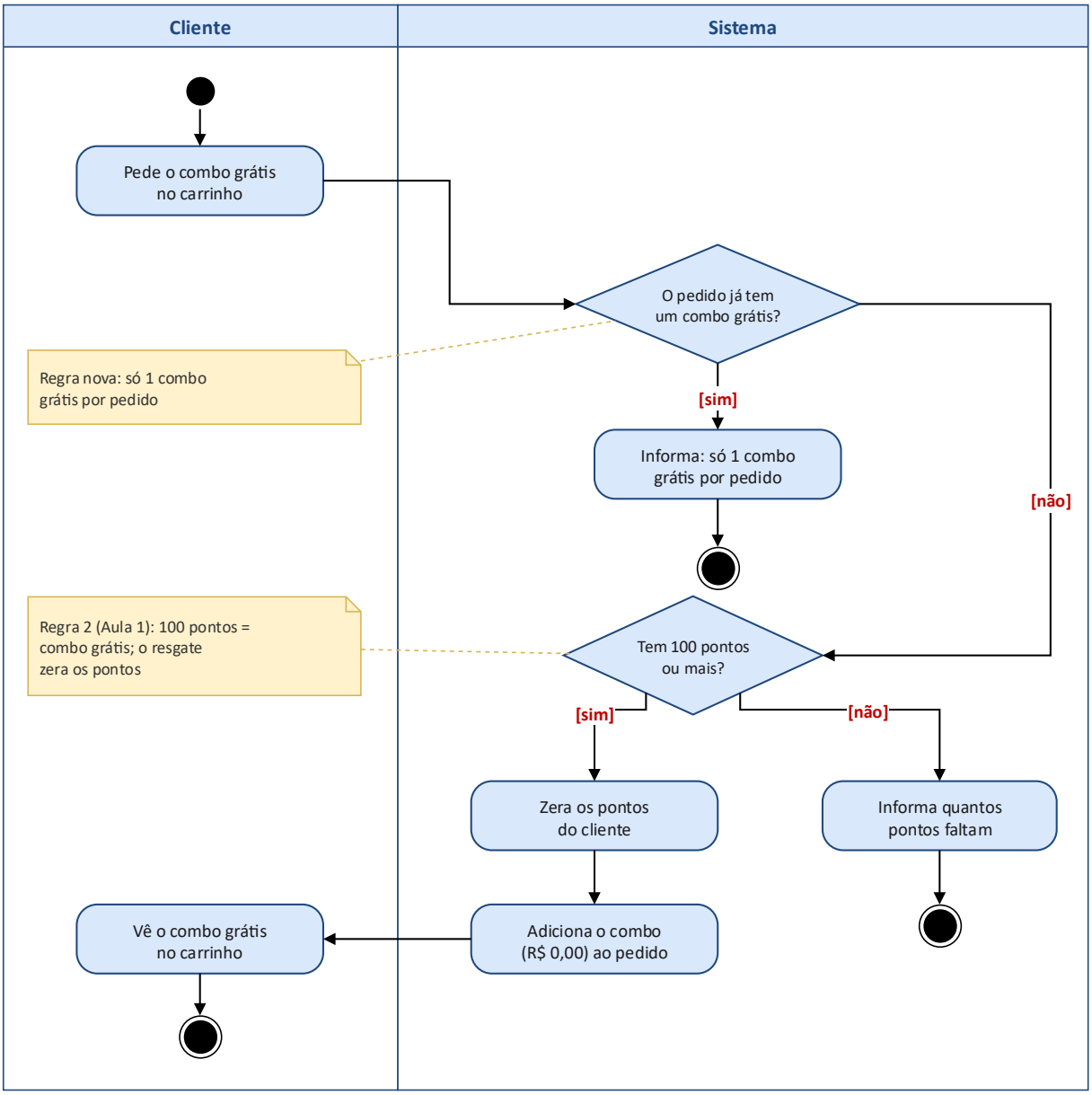


Figura 7 — Resultado do exemplo guiado: atividades “Resgatar combo grátis”.

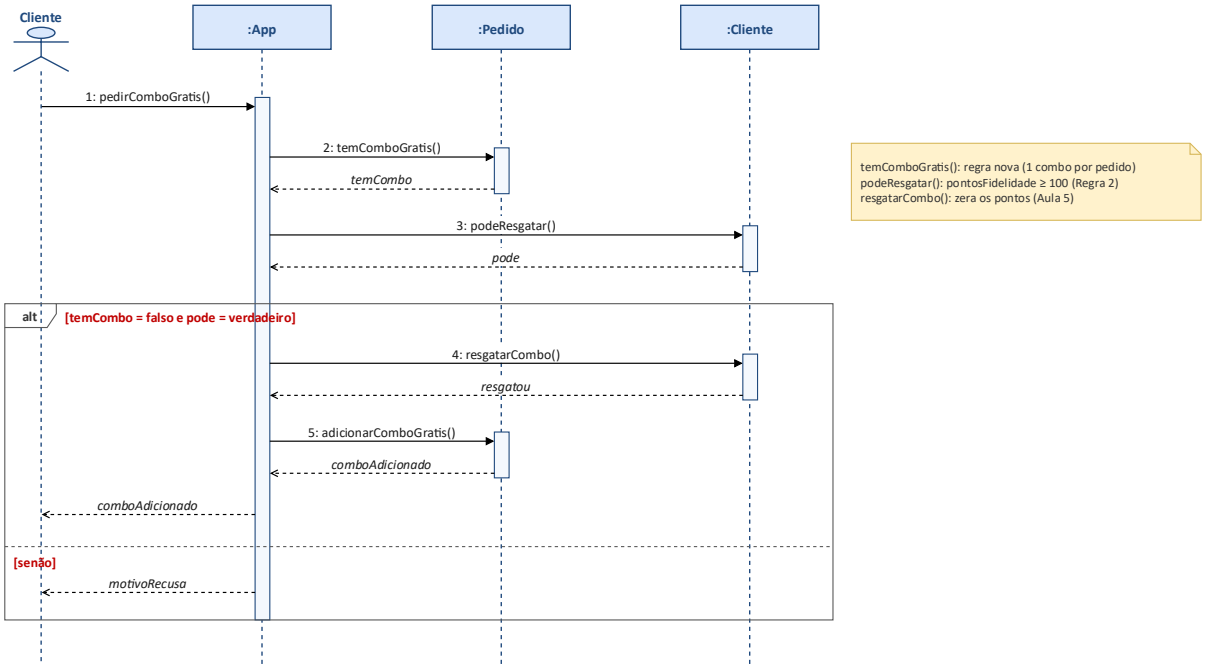


Figura 8 — Resultado do exemplo guiado: sequência "Resgatar combo grátis".

Checklist do exemplo guiado:

- A regra aparece nos 4 diagramas? (caso de uso existente, métodos novos, duas decisões, um alt)
- Os nomes batem? Cliente é ator, raia e participante; :Pedido e :Cliente são as classes Pedido e Cliente.
- temComboGratis(), podeResgatar(), resgatarCombo() e adicionarComboGratis() existem nas classes?
- As duas decisões da atividade viraram UM alt na sequência, com as duas condições na guarda — isso é coerente.
- A ordem é a mesma: primeiro conferir, depois zerar os pontos e adicionar o combo.

5.2 Revisão: o que cada diagrama mostrou

Depois de construir, vale olhar o resultado diagrama por diagrama e lembrar os conceitos principais de cada um (slides 59 a 64).

Casos de uso — O QUE o sistema oferece e PARA QUEM. O ator (boneco) é quem usa o sistema e fica FORA da fronteira; cada caso de uso (elipse) é um objetivo do ator, escrito com verbo; a fronteira (retângulo com o nome do sistema) separa o que é do sistema. No guiado, a regra nova não criou caso de uso: Resgatar Combo Grátis já existia, ligado ao Cliente. O que mudou foi como ele funciona por dentro — e isso aparece nos outros três diagramas.

Classes — QUAIS DADOS existem e COMO se ligam. Cada classe tem nome, atributos (- nome: tipo, os dados que ela guarda, como pontosFidelidade) e métodos (+ nome(), o que ela sabe fazer, como podeResgatar()). As associações ligam as classes, com cardinalidade nas duas pontas: 1 Cliente faz * (muitos) Pedidos. No guiado, a regra nova virou métodos novos — temComboGratis() e adicionarComboGratis() no Pedido, podeResgatar() no Cliente. É o único dos quatro diagramas que é ESTRUTURAL: mostra o que o sistema guarda, não a ordem das coisas.

Atividades — O PASSO A PASSO e QUEM faz cada passo (Figura 7). As raias (Cliente e Sistema) dizem quem executa cada ação; o nó inicial (bolinha preta) e o nó final (alvo) marcam onde o fluxo começa e termina; cada ação é um passo com verbo; cada decisão (losango) é uma pergunta, com as saídas entre colchetes ([sim] / [não]). No guiado, duas decisões geraram três caminhos: o pedido já tem combo, faltam pontos, ou o cliente resgata. As notas amarelas mostram de qual regra de negócio veio cada decisão.

Sequência — QUEM CHAMA QUEM e em que ORDEM (Figura 8). No topo ficam os participantes: o ator Cliente e os objetos :App, :Pedido e :Cliente. Cada mensagem (seta cheia, numerada) chama um método de quem recebe — 2: temComboGratis() é um método do Pedido; cada retorno (seta tracejada) leva a resposta de volta; a ativação (retângulo fino) mostra o tempo em que o objeto está trabalhando. O alt com a guarda [temCombo = falso e pode = verdadeiro] faz o resgate; o [senão] devolve o motivo da recusa. A ordem se lê de cima para baixo: primeiro o app pergunta (mensagens 1 a 3), depois age (4 e 5).

Como os quatro se ligam: o Cliente é ator, raia e participante, com o mesmo nome; temComboGratis() é método do Pedido no diagrama de classes e mensagem 2 na sequência; a decisão [o pedido já tem um combo grátis?] da atividade virou o temCombo da guarda do alt; e a ordem é a mesma nos dois diagramas de comportamento — primeiro conferir, depois zerar os pontos e adicionar o combo. A regra “só 1 combo por pedido” aparece nos quatro: é isso que significa um modelo COERENTE.

6. O draw.io para os 4 diagramas

6.1 Onde fica cada coisa no draw.io

Hoje vocês usam os 4 diagramas no mesmo arquivo: uma página por diagrama (botão “+” na parte de baixo da tela; duplo clique na aba para renomear). Quase todas as formas saem da paleta UML — clique em “UML” no painel da esquerda. Ao parar o mouse sobre um ícone, o draw.io mostra o nome dele em inglês; é esse nome que aparece na tabela abaixo.

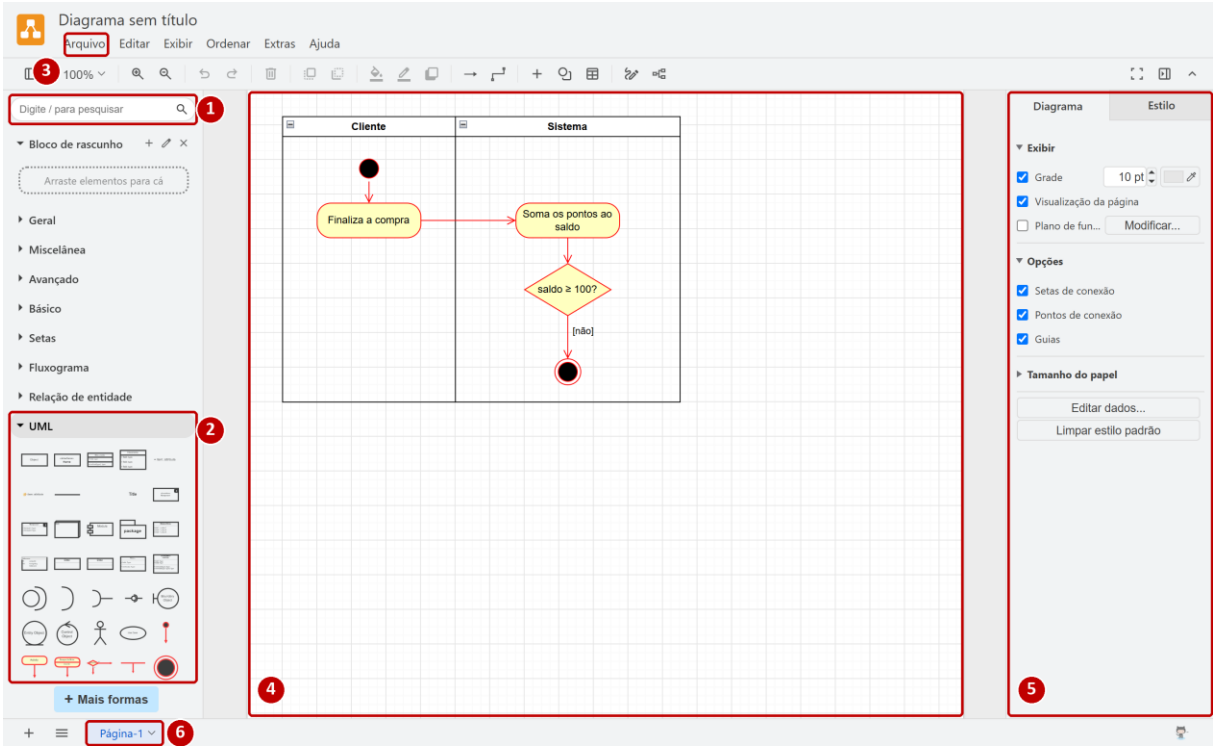


Figura 9 — A tela do draw.io: (1) pesquisa, (2) paleta UML, (3) menu Arquivo, (4) área de desenho, (5) painel de formatação, (6) páginas.



Figura 10 — Casos de uso e classes: as formas ficam no topo da paleta UML.

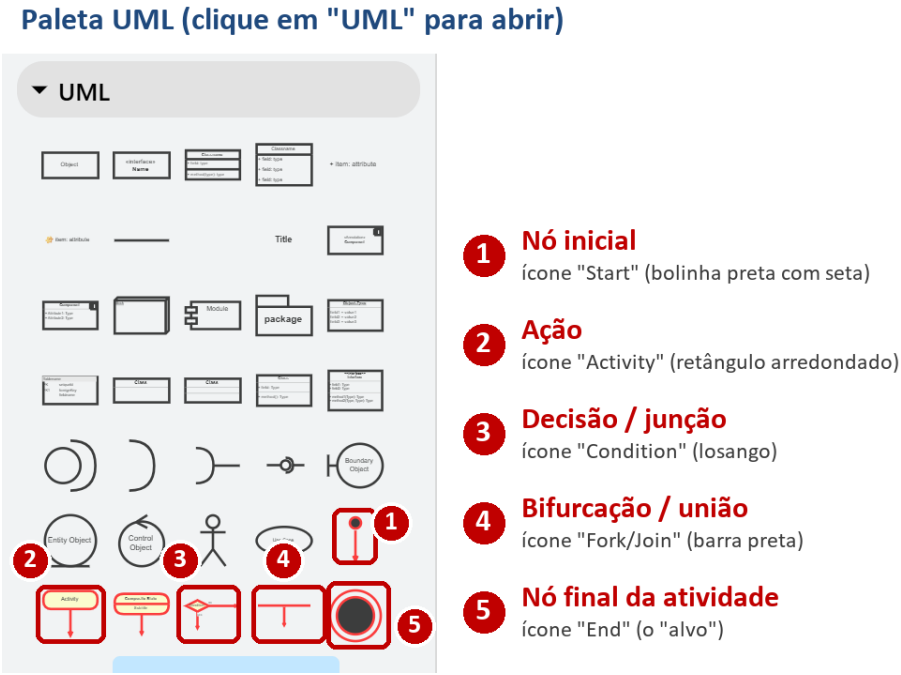


Figura 11 — Atividades: as formas ficam no meio da paleta UML.



Figura 12 — Atividades: a raia (paleta Geral) e o nó final de fluxo (pesquisa “flow final”).

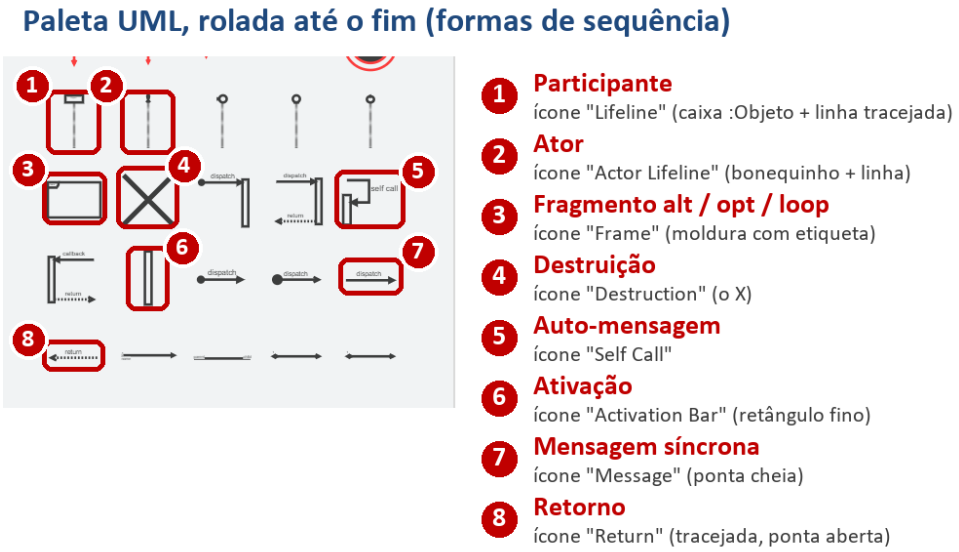


Figura 13 — Sequência: as formas ficam no fim da paleta UML (role a lista).

Diagrama	Elemento	Onde fica no draw.io
Casos de uso	Ator / caso de uso	UML › “Actor” / “Use Case”
Casos de uso	Fronteira do sistema	Geral › “Rectangle”, com o nome do sistema no topo
Casos de uso	«include» / «extend»	UML › “Dependency” (tracejada) — troque o texto “Use”
Classes	Classe	UML › “Class” — duplo clique em cada linha; Ctrl+D numa linha cria outra
Classes	Associação com cardinalidade	UML › “Relation 2” — edite o nome e os rótulos 0..n e 1
Classes	Agregação / composição / herança	UML › “Aggregation 1” / “Composition 1” / “Generalization”
Atividades	Nó inicial / ação / decisão / fim	UML › “Start” / “Activity” / “Condition” / “End”
Atividades	Bifurcação e união / raia	UML › “Fork/Join” / Geral › “Vertical Container”
Sequência	Ator / participante / ativação	UML › “Actor Lifeline” / “Lifeline” / “Activation Bar”
Sequência	Mensagem / retorno / fragmento	UML › “Message” / “Return” /

		“Frame”

Atalhos e cuidados:

- Para ligar duas formas, pare o mouse sobre uma delas e arraste uma das setas azuis até a outra; a seta fica presa quando a forma de destino fica destacada.
- Rótulos, guardas e mensagens: duplo clique na forma ou na seta e digite ([sim], «extend», “1: agendarManutencao(data)”).
- Ctrl+D duplica; Ctrl+Z desfaz; Delete apaga.
- As formas de atividades da paleta UML vêm amarelas com borda vermelha: a cor não muda o significado.

7. Atividade prática 1 — Os 4 diagramas da EcoBike (em grupo)

Em grupos de 3 a 4 pessoas, modelem a EcoBike nos 4 diagramas, no MESMO arquivo, uma página por diagrama. Partam do “Aula 9 - Atividade 1 EcoBike (ponto de partida).drawio”, que já traz as 4 páginas nomeadas. Tempo: 50 minutos, 12 por diagrama. O que mais vale na correção é a COERÊNCIA entre os quatro.

7.1 O cenário EcoBike

A EcoBike é uma loja especializada em venda de bicicletas e serviços de manutenção. Um cliente pode comprar uma bicicleta diretamente na loja ou agendar um serviço de manutenção (revisão, troca de peças, ajuste de marchas). Quando o cliente chega para a manutenção, um mecânico avalia a bicicleta e gera um orçamento; o cliente precisa aprovar o orçamento antes que o serviço comece — se recusar, a ordem de serviço é encerrada. Depois do serviço concluído, o cliente é notificado para retirar a bicicleta e efetua o pagamento na retirada. Cada ordem de serviço é de uma bicicleta e é atendida por um mecânico, e gera no máximo um orçamento e no máximo um pagamento. A gerente da loja acompanha o estoque de peças e de bicicletas disponíveis para venda.

7.2 Cronograma

Tempo	O que fazer
0 a 2 min	Grifos no texto e lista de nomes combinados (atores, classes) na página 1
2 a 14 min	Diagrama de casos de uso
14 a 26 min	Diagrama de classes, com cardinalidade
26 a 38 min	Diagrama de atividades, do agendamento até a retirada, com pelo menos 1 decisão
38 a 50 min	Diagrama de sequência do mesmo fluxo + conferência de coerência

No draw.io, para esta atividade (tabela da seção 6.1):

- Arquivo › Abrir: o ponto de partida já tem as 4 páginas nomeadas e a fronteira “EcoBike” na página 1.
- Casos de uso: “Actor” e “Use Case” (paleta UML).
- Classes: “Class” e “Relation 2” (cardinalidade nas duas pontas).
- Atividades: raias com “Vertical Container” (paleta Geral); “Start”, “Activity”, “Condition”, “End”.
- Sequência: “Actor Lifeline”, “Lifeline”, “Message”, “Return” e “Frame”.

Pistas para o grupo

- Substantivos que viram classes: cliente, bicicleta, ordem de serviço, orçamento, mecânico, pagamento.
- “precisa APROVAR antes” é uma decisão na atividade e um passo obrigatório na sequência.
- “UMA bicicleta”, “UM mecânico”, “no máximo um”: as cardinalidades estão no texto.
- Cliente é ator (a pessoa) e também classe (os dados dela).
- Casos de uso começam com verbo: Agendar Manutenção, Aprovar Orçamento...
- “Avaliar SEMPRE gera orçamento”: o que sempre acontece é «include»; o que só às vezes, «extend»
- Mensagem para um objeto :X = método da classe X (ex.: aprovar() de Orcamento)

Checklist da atividade 1

- 4 diagramas, uma página cada, dentro do tempo.
- Os mesmos nomes em todos: ator = raia = participante; classe = participante.
- Cardinalidades justificadas pelo texto.
- Decisão [cliente aprova o orçamento?] com as duas saídas na atividade.
- Mensagens da sequência na mesma ordem das ações da atividade.

Dicas para o grupo

- Um lê o texto e controla o tempo; os outros desenham — e troquem a cada diagrama.
- Acabou o tempo? Passem para o próximo: incompleto e coerente vale mais que metade perfeita.
- Deixem na página 1 a lista de nomes combinados e usem SEMPRE ela.
- Nos 2 minutos finais, leiam a atividade e a sequência lado a lado.

8. Exercício: encontre as incoerências

Em duplas, analisem a figura abaixo: as regras do TechBurger, um pedaço do diagrama de classes e o diagrama de sequência “Pagar o pedido”. Há 6 incoerências — entre os dois diagramas, ou entre um diagrama e a regra. Para cada uma: (1) marquem onde está, (2) digam QUAL regra de coerência da seção 4 foi violada e (3) escrevam a correção. Tentem primeiro sem consultar o resto da apostila. Respostas no Gabarito.

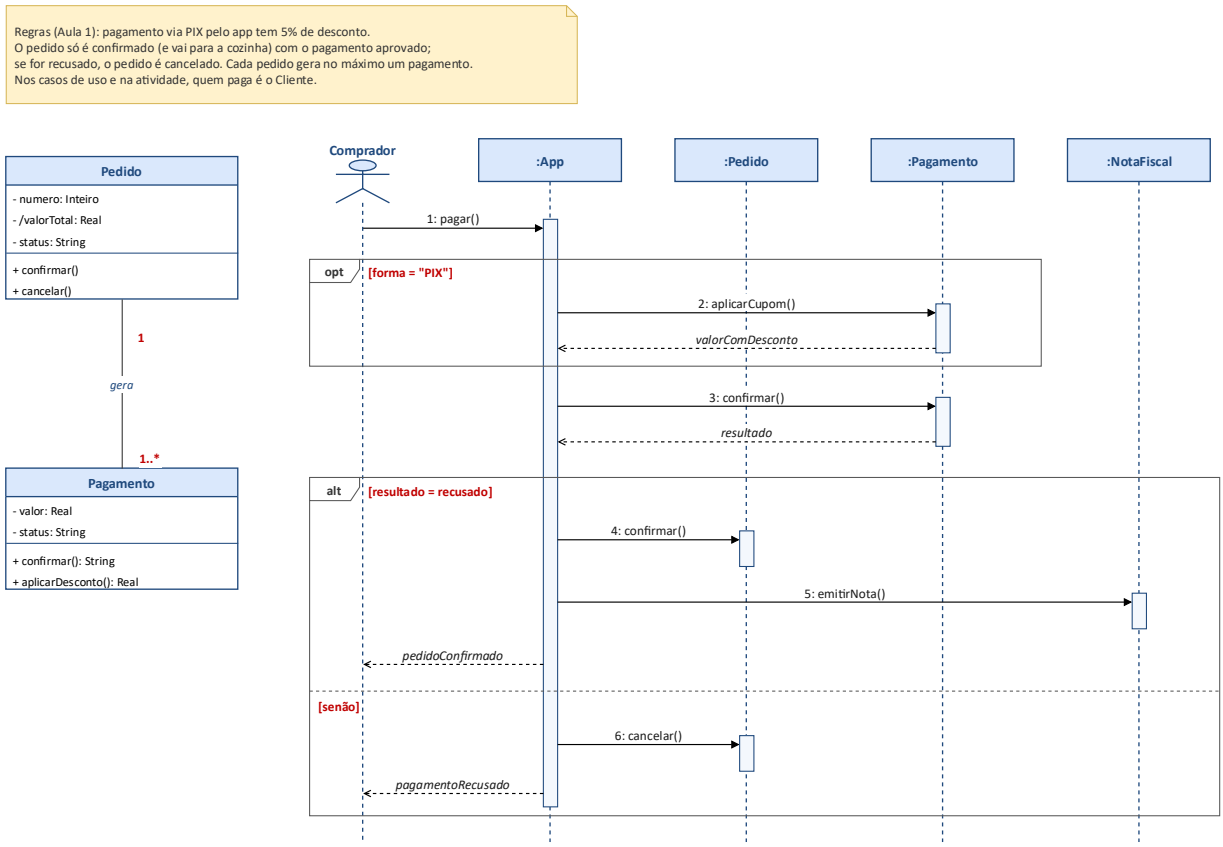


Figura 14 — Classes x sequência "Pagar o pedido": 6 incoerências.

Dica: comparem cada participante com as classes, cada mensagem com os métodos, cada guarda com os atributos e com a regra, cada cardinalidade com as regras e cada ator com os outros diagramas do TechBurger (seção 3).

9. Erros comuns ao integrar os diagramas

- Desenhar cada diagrama do zero, sem olhar os outros.
- Mudar um diagrama e esquecer de atualizar os outros.
- Inventar classes e métodos que o texto não pede — e esquecer os que ele pede.
- Deixar a regra de negócio só na cabeça: ela precisa estar nos modelos.
- Achar que a sequência precisa mostrar todos os caminhos da atividade (ela mostra um cenário).
- Gastar todo o tempo no primeiro diagrama e não fazer os outros.

10. Boas práticas

- Comecem pelas regras de negócio e combinem os nomes antes de desenhar.
- Um arquivo, uma página por diagrama, os mesmos nomes em todos.
- A cada diagrama novo, revisem os anteriores.
- Usem a matriz de rastreabilidade como checklist final.
- Prefiram um modelo simples e coerente a um grande e contraditório.

Glossário

Termo	Significado
Vista	Um jeito de olhar o sistema. Cada diagrama UML é uma vista do mesmo sistema.
Coerência	Todos os diagramas contam a mesma história, com os mesmos nomes, números e ordem.
Incoerência	Um diagrama diz algo DIFERENTE do outro (mostrar menos não é incoerência).
Rastreabilidade	Conseguir seguir uma regra ou requisito do texto até cada diagrama (e, depois, até o código e os testes).
Matriz de rastreabilidade	Tabela com uma linha por regra e uma coluna por artefato; célula vazia = lacuna.
Lacuna	Regra que não aparece num diagrama onde deveria aparecer.
Técnica dos grifos	Marcar no texto os substantivos, verbos, condições, repetições e números.
Modelagem iterativa	Desenhar, conferir e corrigir em ciclos: cada diagrama novo revisa os anteriores.

Perguntas de revisão

1. Por que um sistema precisa de mais de um tipo de diagrama?
2. Qual a pergunta que cada um dos 4 diagramas responde?
3. Cite 3 regras de coerência entre o diagrama de classes e o de sequência.
4. Como um «extend» do caso de uso costuma aparecer na sequência?
5. O que é uma matriz de rastreabilidade? O que significa uma célula vazia?
6. A sequência mostra menos caminhos que a atividade. Isso é incoerência? Por quê?
7. Na técnica dos grifos, o que viram os substantivos, os verbos e os “se”?
8. Dê um exemplo de regra de negócio que vira cardinalidade e um de regra que vira decisão.

Quiz relâmpago para casa

Cinco perguntas para conferir o que ficou. Respostas no Gabarito.

1. Quais são os 4 diagramas UML do curso? Qual deles é estrutural?
2. O ator do caso de uso aparece como o quê na atividade? E na sequência?
3. aplicarCupom() chega em :Pagamento, que só tem aplicarDesconto(). O que está errado?
4. O que significa uma célula vazia na matriz de rastreabilidade?
5. Por onde se começa a modelar: pelas classes ou pelas regras de negócio?